

Solving the Korteweg–de Vries Equation using Physics-Informed Neural Networks: A Systematic Study of Data, Noise, and Physics Constraints

Soham Pujari (2024A7PS0490G) Nirek Agarwal (2024A7PS0581G)

BITS Pilani, Goa Campus

Preliminary work under **Prof. Snehanshu Saha**

April 8, 2026

Abstract

We implement a Physics-Informed Neural Network (PINN) to solve the Korteweg–de Vries (KdV) equation, achieving a relative L^2 error of 0.0717% on the single-soliton solution. Going beyond this baseline, we run four experiments that reveal how PINNs actually behave when things get messy—noisy sensors, missing initial conditions, and harder physics. Our main findings: (1) the PDE residual acts as a noise filter, cutting error by $4.5\times$ over a data-only network at 10% noise; (2) L-BFGS fine-tuning, widely recommended in the literature, actually *hurts* PINN accuracy when data is noisy; (3) 200 scattered measurements can fully replace the initial condition; and (4) the same architecture that handles $c = 2$ fails at $c = 4$ due to steeper gradients and boundary effects. These are not KdV-specific findings—they are practical guidelines for anyone deploying PINNs on wave-like PDEs.

Contents

1	Introduction	2
2	Problem Setup	2
2.1	The KdV Equation	2
2.2	Exact Solution	3
2.3	Initial and Boundary Conditions	3
3	Network and Training	3
3.1	Architecture	3
3.2	Loss Function	4
3.3	Derivative Sanity Check	5
3.4	Two-Phase Training	5
4	Baseline Results	5
4.1	Training Convergence	5
4.2	Solution Quality	6
4.3	Conservation of Mass	7
5	Experiments	8
5.1	Experiment 1: What Happens When Data is Noisy?	8
5.2	Experiment 2: How Many Collocation Points Do We Need?	10
5.3	Experiment 3: Can We Drop the Initial Condition?	11
5.4	Experiment 4: Faster and Slower Waves	12
6	Discussion: What This Tells Us About Using PINNs	12
7	Conclusion	13

1 Introduction

The Korteweg–de Vries (KdV) equation is a nonlinear PDE that models shallow water wave propagation. Its defining feature is the soliton—a localised wave that travels without changing shape, sustained by a balance between nonlinear steepening and dispersive spreading [1]. First observed experimentally by John Scott Russell in 1834 and formalised mathematically by Korteweg and de Vries in 1895, the soliton remains a central object of study in nonlinear dynamics [4].

Classical numerical methods (finite differences, finite elements) can solve the KdV equation, but the third-order spatial derivative makes them expensive: stability constraints are tight, numerical dispersion accumulates, and long-time tracking of the wave requires fine grids. The KdV equation is therefore a natural testbed for Physics-Informed Neural Networks (PINNs) [2], which sidestep grid-based discretisation entirely by training a neural network to satisfy the PDE as a soft constraint.

Most PINN papers on KdV stop at demonstrating that the method works: train a network, show it matches the exact solution, report the L^2 error, and move on. That is necessary but not sufficient. The real questions for anyone who wants to *use* PINNs in practice are different: What happens when my sensor data is noisy? How many collocation points do I actually need? Do I need to know the initial condition, or can sparse measurements substitute? Will the same setup work if the wave is faster or steeper?

This paper answers those questions. We first build a high-accuracy baseline ($L^2 = 0.0717\%$), then run four controlled experiments that systematically vary noise level, PDE enforcement density, constraint composition, and wave speed. The results expose concrete guidelines—including a counterintuitive finding about the L-BFGS optimizer—that apply to any PDE with localised propagating features, not just KdV.

All code is available at <https://github.com/sohp2005/KdV-solution-using-PINNs.git>.

2 Problem Setup

2.1 The KdV Equation

We use the dimensionless KdV equation with the negative sign convention ($u \mapsto -u$, following Section 4 of [1]):

$$\frac{\partial u}{\partial t} - 6u \frac{\partial u}{\partial x} + \frac{\partial^3 u}{\partial x^3} = 0, \quad (x, t) \in \Omega = [-10, 20] \times [0, 5]. \quad (1)$$

Under this convention the soliton appears as a trough (negative amplitude) rather than a peak. This is physically equivalent to the standard form via coordinate inversion.

The domain Ω is asymmetric by design. At speed $c = 2$, the soliton starts at $x = 0$ and reaches $x \approx 10$ by $t = 5$, so the right boundary at $x = 20$ gives a comfortable buffer. We avoid extending further (e.g., $x = 100$) because a larger domain dilutes the randomly sampled collocation points into featureless flat regions, making it harder for the network to resolve the sharp soliton peak.

2.2 Exact Solution

Equation (1) admits the travelling wave:

$$u(x, t) = \frac{-c}{2 \cosh^2\left(\frac{\sqrt{c}}{2}(x - ct)\right)}, \quad (2)$$

where c is the wave speed. We use $c = 2$ as our default, giving a peak depth of -1 .

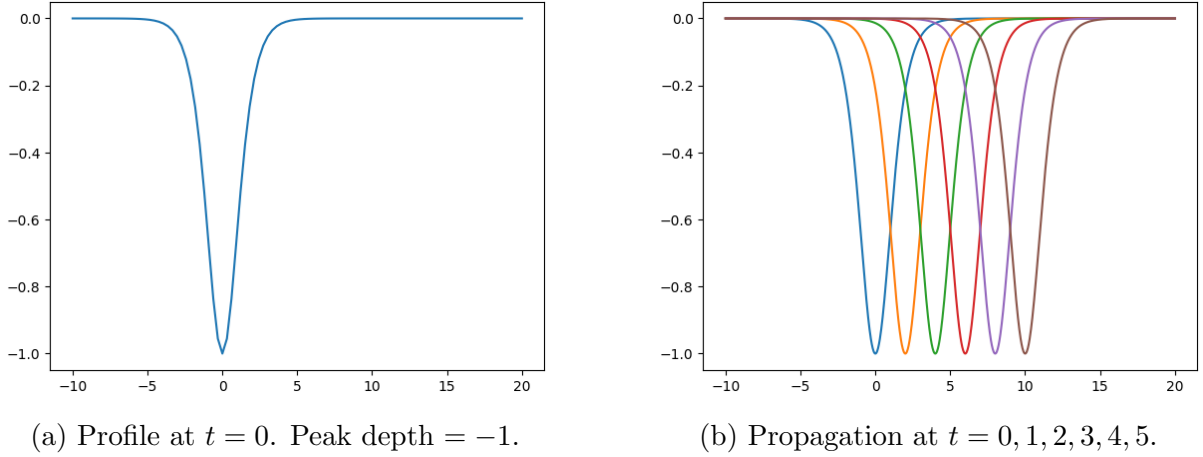


Figure 1: Exact soliton with $c = 2$, verifying Eq. (2).

2.3 Initial and Boundary Conditions

The initial condition is the soliton at $t = 0$:

$$u(x, 0) = \frac{-2}{2 \cosh^2\left(\frac{\sqrt{2}}{2}x\right)}, \quad x \in [-10, 20]. \quad (3)$$

The soliton decays exponentially, so the boundaries are effectively zero:

$$u(-10, t) \approx 0, \quad u(20, t) \approx 0, \quad t \in [0, 5]. \quad (4)$$

3 Network and Training

3.1 Architecture

We use a fully connected feedforward network: 2 inputs (x, t) , six hidden layers of 50 neurons each with tanh activation, and 1 output $u_\theta(x, t)$. Weights are initialised with Xavier normal; biases are zero.

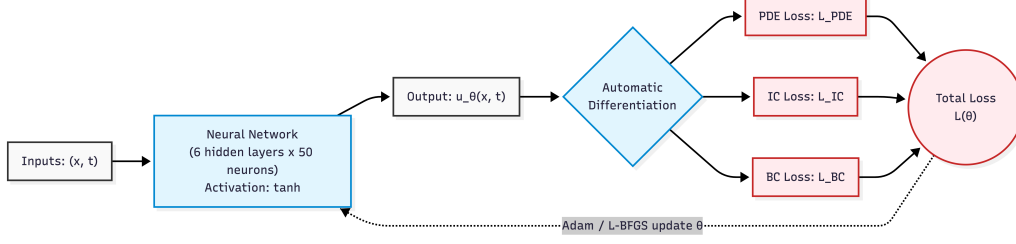


Figure 2: PINN schematic. The network outputs $u_\theta(x, t)$; automatic differentiation computes the PDE residual, and the total loss combines physics, boundary, and (optionally) data terms.

The choice of tanh is not cosmetic. The KdV equation requires third-order derivatives $\partial^3 u_\theta / \partial x^3$ —ReLU has discontinuous second derivatives, and Sigmoid is strictly positive, which contradicts the wave’s natural amplitude range. Only tanh (or similar smooth activations) gives well-defined gradients at all orders.

3.2 Loss Function

The total loss can include up to four terms:

$$\mathcal{L}(\theta) = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} + \lambda_{\text{data}} \mathcal{L}_{\text{data}}. \quad (5)$$

Each λ is either 0 or 1 depending on the experiment (Table 1).

Table 1: Which loss terms are active in each configuration.

Configuration	$\mathcal{L}_{\text{PDE}}$	$\mathcal{L}_{\text{IC}}$	$\mathcal{L}_{\text{BC}}$	$\mathcal{L}_{\text{data}}$
Baseline PINN	✓	✓	✓	✓
Data-only NN	—	—	—	✓
No-IC PINN	✓	—	✓	✓

The individual terms are:

PDE residual ($N_r = 10,000$ random points across Ω):

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_r} \sum_{i=1}^{N_r} \left(\left. \frac{\partial u_\theta}{\partial t} \right|_{x_i, t_i} - 6 u_\theta \left. \frac{\partial u_\theta}{\partial x} \right|_{x_i, t_i} + \left. \frac{\partial^3 u_\theta}{\partial x^3} \right|_{x_i, t_i} \right)^2. \quad (6)$$

All derivatives are computed exactly via PyTorch autograd.

Initial condition ($N_{\text{IC}} = 500$ points along $t = 0$):

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{\text{IC}}} \sum_{i=1}^{N_{\text{IC}}} (u_\theta(x_i, 0) - u(x_i, 0))^2. \quad (7)$$

Boundary condition ($N_{\text{BC}} = 100$ points per edge):

$$\mathcal{L}_{\text{BC}} = \frac{1}{2N_{\text{BC}}} \sum_{i=1}^{N_{\text{BC}}} [u_\theta(-10, t_i)^2 + u_\theta(20, t_i)^2]. \quad (8)$$

Data loss ($N_{\text{data}} = 200$ scattered points, optionally noisy):

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} (u_{\theta}(x_i, t_i) - \tilde{u}_i)^2, \quad \tilde{u}_i = u(x_i, t_i) + \sigma \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, 1). \quad (9)$$

Here σ is the noise level as a fraction of the soliton amplitude (which is $|c/2| = 1$ for $c = 2$).

3.3 Derivative Sanity Check

Before training, we verified the autograd pipeline by comparing $\partial u / \partial x$ from autograd against centred finite differences ($h = 10^{-4}$) on 100 points over $[-5, 5]$. The maximum discrepancy was ≈ 0.001 , confirming the computational graph is correct.

```
#!/usr/bin/env python
# check whether derivatives function works by just calc derivatives the f(x+h)-f(x-h)/2h vala defintion and compare with torch ka grad
x = torch.linspace(-5, 5, 100, requires_grad=True) #this requires grad=True stores the value so that we can calc its derivative in the future and the
t = torch.zeros_like(x)
c = 2.0

u = soliton_solution(x, t, c)
u_x_auto = torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u), create_graph=True)[0]

h = 0.0001
u_plus = soliton_solution(x + h, t, c)
u_minus = soliton_solution(x - h, t, c)
u_x_finite = (u_plus - u_minus) / (2 * h)

print("Autograd gives:", u_x_auto[5].detach().numpy())
print("Finite diff gives:", u_x_finite[5].detach().numpy())
print("Max difference:", (u_x_auto - u_x_finite).abs().max().item())

Autograd gives: [-0.00478822 -0.00552061 -0.0063645  -0.00733672 -0.00845655]
Finite diff gives: [-0.00479515 -0.00552973 -0.0063749  -0.00734348 -0.00847038]
Max difference: 0.0009848922491073608
```

Figure 3: Autograd vs. finite-difference check for $\partial u / \partial x$. Max difference ≈ 0.001 .

3.4 Two-Phase Training

We train in two phases:

Phase 1: Adam (15,000 iterations, $\text{lr} = 10^{-3}$). This gets us into the right neighbourhood quickly, but the loss oscillates around $\sim 10^{-5}$ without settling.

Phase 2: L-BFGS (2,000 iterations, $\text{max_iter}=20$, $\text{history_size}=50$). The quasi-Newton curvature information drives the loss down to $\sim 10^{-7}$ and stabilises it.

As we will show in Section 5.1, Phase 2 is beneficial for clean data but can backfire when the data is noisy.

4 Baseline Results

With all four loss terms active and clean data ($\sigma = 0$), the network converges to a relative L^2 error of **0.0717%**.

4.1 Training Convergence

Figure 4 shows the Adam phase: all loss components drop from ~ 0.64 to $\sim 10^{-5}$ at comparable rates, confirming that equal weights work fine for this problem. L-BFGS then locks the total loss at 3.95×10^{-7} .

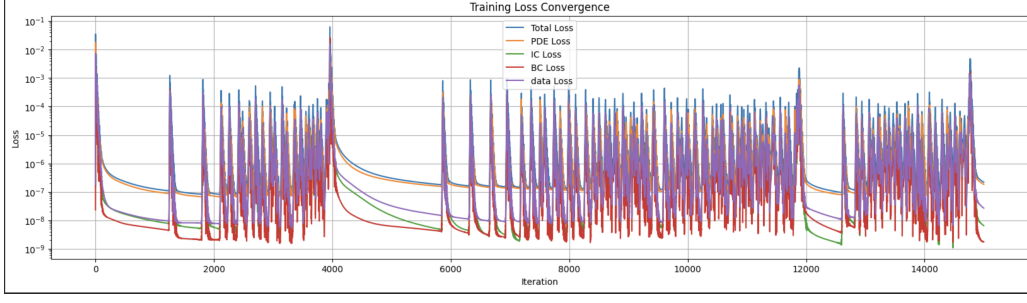


Figure 4: Training loss (log scale) over 15,000 Adam iterations. All components decrease together; oscillations near 10^{-5} motivate the switch to L-BFGS.

Iter	0	Total: 0.641338	PDE: 0.035201	IC: 0.346590	BC: 0.259546
Iter	500	Total: 0.000477	PDE: 0.000420	IC: 0.000052	BC: 0.000004
Iter	1000	Total: 0.000105	PDE: 0.000096	IC: 0.000008	BC: 0.000001
Iter	1500	Total: 0.000082	PDE: 0.000046	IC: 0.000017	BC: 0.000018
Iter	2000	Total: 0.000018	PDE: 0.000017	IC: 0.000001	BC: 0.000000
Iter	2500	Total: 0.000019	PDE: 0.000016	IC: 0.000002	BC: 0.000001
Iter	3000	Total: 0.000010	PDE: 0.000009	IC: 0.000001	BC: 0.000001
Iter	3500	Total: 0.000007	PDE: 0.000007	IC: 0.000000	BC: 0.000000
Iter	4000	Total: 0.000008	PDE: 0.000007	IC: 0.000000	BC: 0.000000
Iter	4500	Total: 0.000067	PDE: 0.000030	IC: 0.000019	BC: 0.000018
Iter	5000	Total: 0.000009	PDE: 0.000005	IC: 0.000002	BC: 0.000002
Iter	5500	Total: 0.000011	PDE: 0.000005	IC: 0.000003	BC: 0.000003
Iter	6000	Total: 0.000122	PDE: 0.000018	IC: 0.000049	BC: 0.000054
Iter	6500	Total: 0.000070	PDE: 0.000056	IC: 0.000011	BC: 0.000002
Iter	7000	Total: 0.000057	PDE: 0.000052	IC: 0.000004	BC: 0.000001
Iter	7500	Total: 0.000244	PDE: 0.000214	IC: 0.000024	BC: 0.000006
Iter	8000	Total: 0.000140	PDE: 0.000025	IC: 0.000053	BC: 0.000062
Iter	8500	Total: 0.000004	PDE: 0.000002	IC: 0.000001	BC: 0.000001
Iter	9000	Total: 0.000016	PDE: 0.000008	IC: 0.000004	BC: 0.000005
Iter	9500	Total: 0.000004	PDE: 0.000001	IC: 0.000001	BC: 0.000001
Iter	10000	Total: 0.000226	PDE: 0.000059	IC: 0.000078	BC: 0.000088
Iter	10500	Total: 0.000032	PDE: 0.000014	IC: 0.000010	BC: 0.000008
Iter	11000	Total: 0.000011	PDE: 0.000005	IC: 0.000004	BC: 0.000003
Iter	11500	Total: 0.000011	PDE: 0.000010	IC: 0.000001	BC: 0.000000
Iter	12000	Total: 0.000020	PDE: 0.000010	IC: 0.000005	BC: 0.000005
Iter	12500	Total: 0.000032	PDE: 0.000027	IC: 0.000004	BC: 0.000001
Iter	13000	Total: 0.000120	PDE: 0.000105	IC: 0.000013	BC: 0.000002
Iter	13500	Total: 0.000005	PDE: 0.000004	IC: 0.000001	BC: 0.000001
Iter	14000	Total: 0.000005	PDE: 0.000004	IC: 0.000000	BC: 0.000000
Iter	14500	Total: 0.000041	PDE: 0.000017	IC: 0.000011	BC: 0.000012

(a) Adam (logged every 500 iter.)

LBFGS 0	Total: 0.0000045643
LBFGS 100	Total: 0.0000003947
LBFGS 200	Total: 0.0000003947
LBFGS 300	Total: 0.0000003947
LBFGS 400	Total: 0.0000003947
LBFGS 500	Total: 0.0000003947
LBFGS 600	Total: 0.0000003947
LBFGS 700	Total: 0.0000003947
LBFGS 800	Total: 0.0000003947
LBFGS 900	Total: 0.0000003947
LBFGS 1000	Total: 0.0000003947
LBFGS 1100	Total: 0.0000003947
LBFGS 1200	Total: 0.0000003947
LBFGS 1300	Total: 0.0000003947
LBFGS 1400	Total: 0.0000003947
LBFGS 1500	Total: 0.0000003947
LBFGS 1600	Total: 0.0000003947
LBFGS 1700	Total: 0.0000003947
LBFGS 1800	Total: 0.0000003947
LBFGS 1900	Total: 0.0000003947

(b) L-BFGS (logged every 100 iter.)

Figure 5: Console output during training. L-BFGS stabilises the loss at 3.95×10^{-7} almost immediately.

4.2 Solution Quality

The predicted soliton is visually indistinguishable from the exact solution at every time step (Figure 6). The error heatmap (Figure 7) confirms that residual error is concentrated along the soliton trajectory and grows slightly at later times—a natural consequence of error accumulation during propagation.

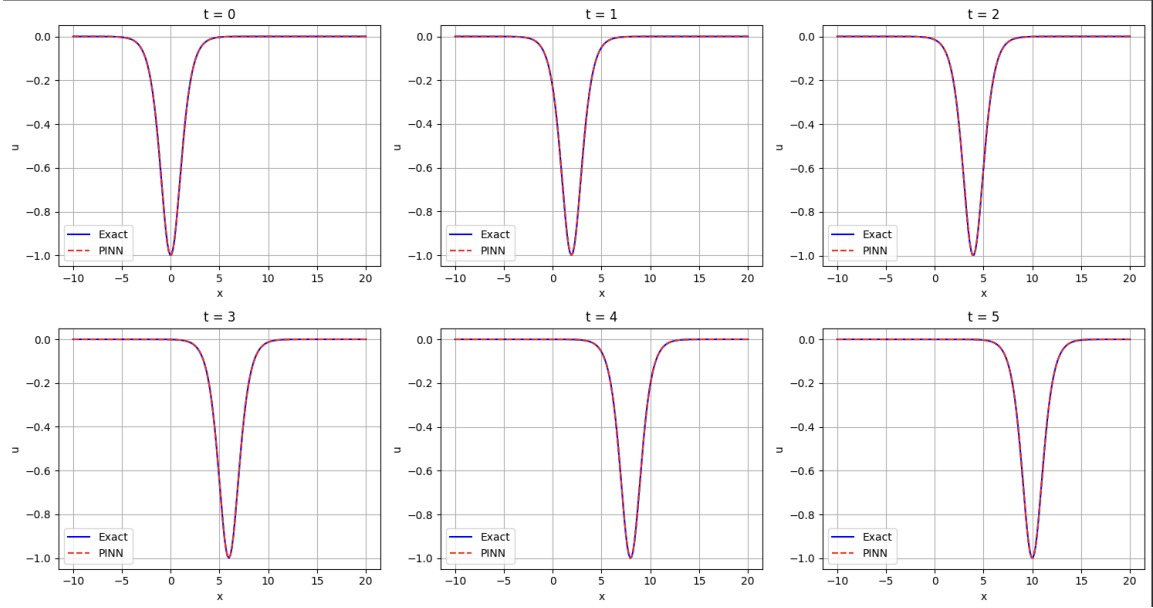


Figure 6: PINN prediction (red dashed) vs. exact soliton (blue solid) at $t = 0, 1, \dots, 5$.

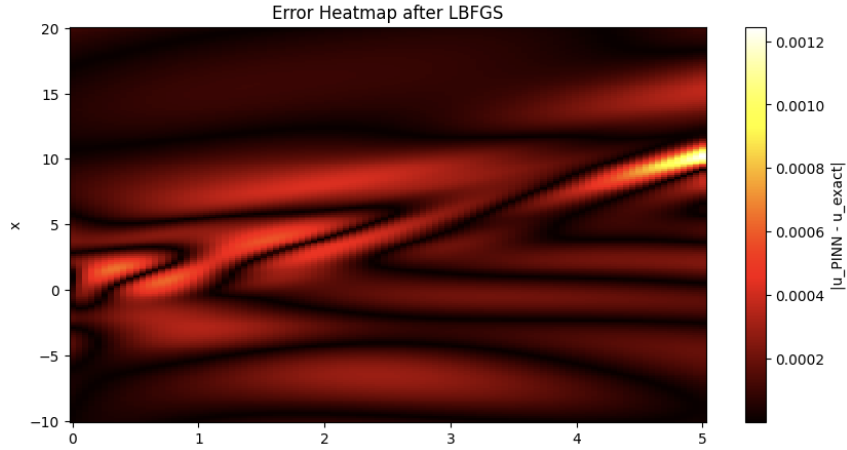


Figure 7: Pointwise absolute error $|u_\theta - u_{\text{exact}}|$. Error tracks the soliton path and widens at later times.

4.3 Conservation of Mass

The KdV equation conserves mass: $\frac{d}{dt} \int u \, dx = 0$ [1]. We never enforced this—it is not part of the loss. Yet the network respects it: the integral $\int u_\theta \, dx$ stays within $[-2.829, -2.820]$ across all time steps, a variation of $< 0.4\%$. This is strong evidence that the network has internalised the wave dynamics, not just memorised boundary values.

Table 2: Mass conservation check ($\int u_\theta dx$ via trapezoidal rule).

t	$\int u_\theta dx$
0	-2.828829
1	-2.825412
2	-2.826964
3	-2.824013
4	-2.820568
5	-2.822070

5 Experiments

The baseline shows PINNs work. The harder question is: *when do they break, and what can we learn from how they break?* We designed four experiments, each varying one thing while holding everything else fixed.

5.1 Experiment 1: What Happens When Data is Noisy?

We corrupt the 200 data points with Gaussian noise ($\sigma = 1\%, 5\%, 10\%$ of the soliton amplitude) and train two models on the exact same noisy data: the full PINN (with physics) and a data-only NN (no PDE, no IC, no BC—just curve fitting).

Table 3: L^2 error (%) under noise. Both after-Adam and after-L-BFGS results are shown.

Noise	PINN		Data-only NN	
	After Adam	After L-BFGS	After Adam	After L-BFGS
0%	0.0717	0.0717	1.198	1.198
1%	0.825	1.130	11.07	11.07
5%	5.514	6.778	42.83	38.81
10%	12.16	14.37	54.70	59.15

The visual difference at 10% noise is striking. Figure 8 shows the PINN: it tracks the soliton’s shape and position correctly, though with some amplitude error and spurious ripples. Figure 9 shows the data-only network on the same data: it has no idea what a wave looks like and produces wild oscillations everywhere.

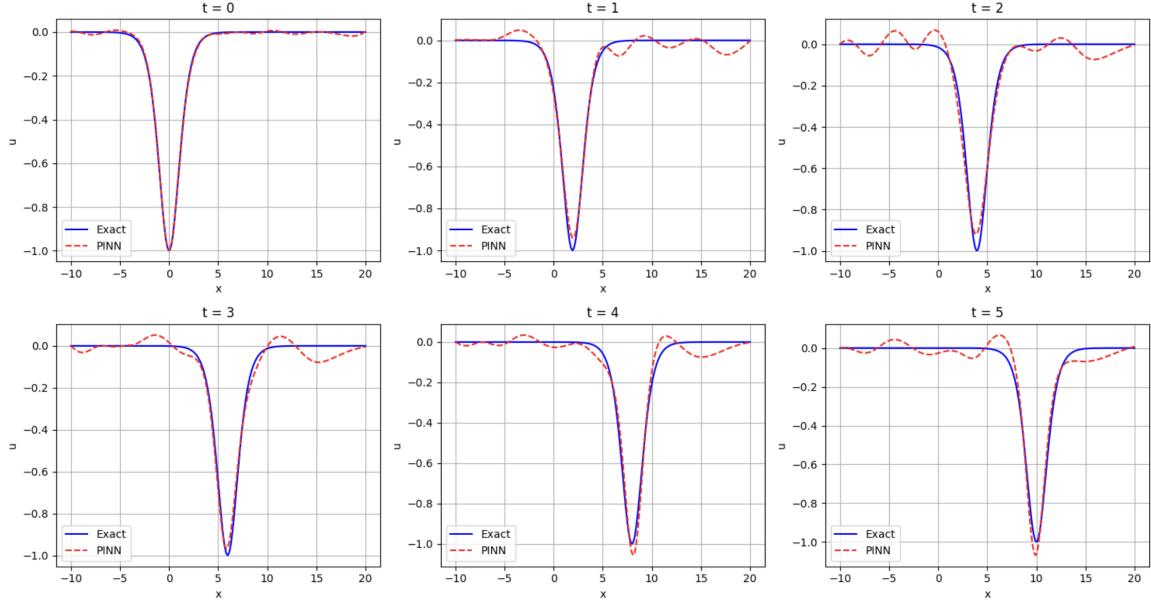


Figure 8: PINN with 10% noise. The wave shape is recovered despite substantial corruption.

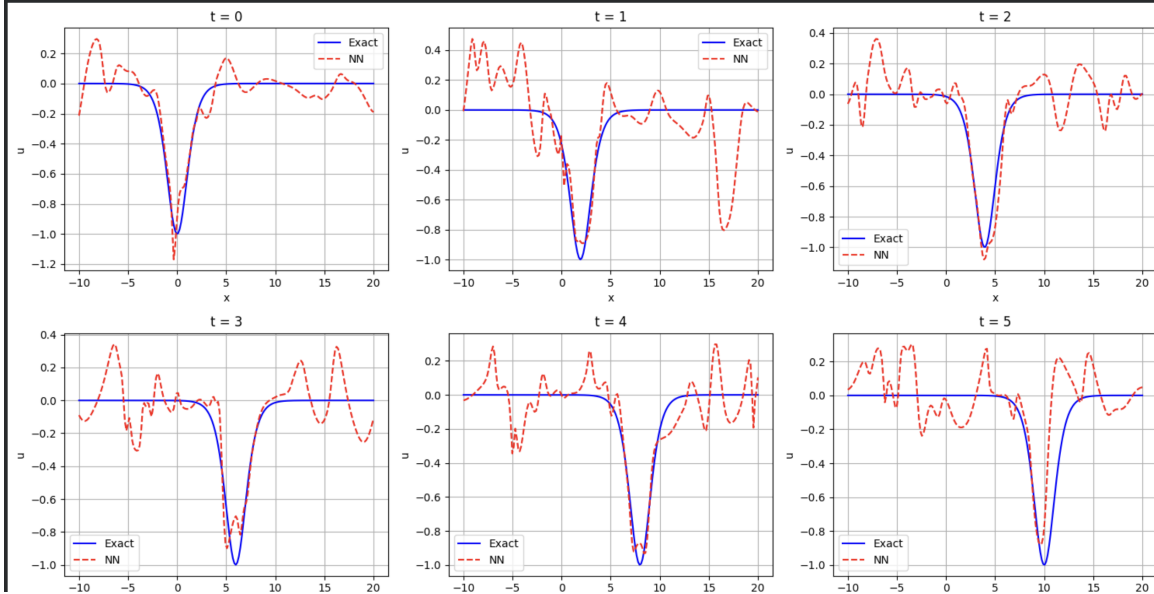
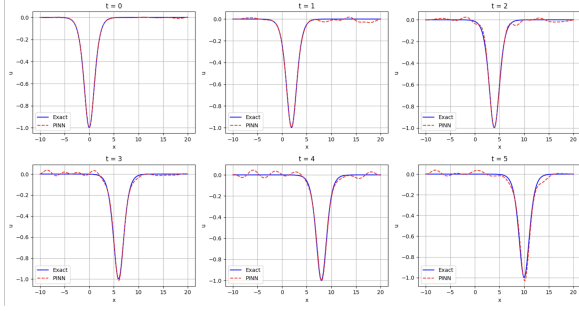
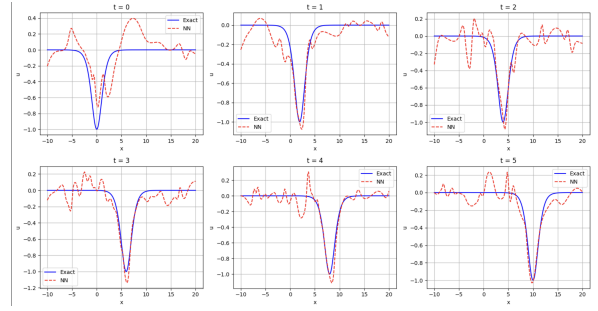


Figure 9: Data-only NN with the same 10% noisy data. Without physics, the network has nothing to regularise against and overfits the noise completely.

We also show the 5% noise cases for reference:



(a) PINN at 5% noise.



(b) Data-only NN at 5% noise.

Figure 10: At 5% noise the PINN is visibly better; the data-only model already shows significant distortion.

What we learned:

The PDE acts as a regulariser. At 10% noise, the PINN’s error (12.2%) is $4.5\times$ lower than the data-only model (54.7%). The physics constrains the network to produce solutions that look like KdV waves, so it cannot overfit random noise the way an unconstrained network does.

But the more surprising finding is in the optimizer column. Look at the PINN rows: L-BFGS makes things *worse* at every noise level above zero. At 5% noise, the error goes from 5.5% after Adam to 6.8% after L-BFGS. For the data-only model, L-BFGS either helps or has no effect. Why?

L-BFGS uses curvature information to converge aggressively. In a PINN, the PDE loss and the noisy data loss pull in different directions—the PDE says “be smooth,” the noisy data says “hit these corrupted targets.” Adam, being a first-order method, navigates this tension with smaller, more cautious steps. L-BFGS charges ahead and overfits the noise. The practical takeaway: **when data is noisy, skip L-BFGS or stop it early.**

5.2 Experiment 2: How Many Collocation Points Do We Need?

We vary N_r from 500 to 20,000, keeping everything else fixed (IC, BC, data, architecture, training schedule).

Table 4: L^2 error vs. number of collocation points.

N_r	L^2 error (%)
500	0.2108
1,000	0.0957
2,500	0.0822
5,000	0.0797
10,000	0.0679
15,000	0.0691
20,000	0.0736

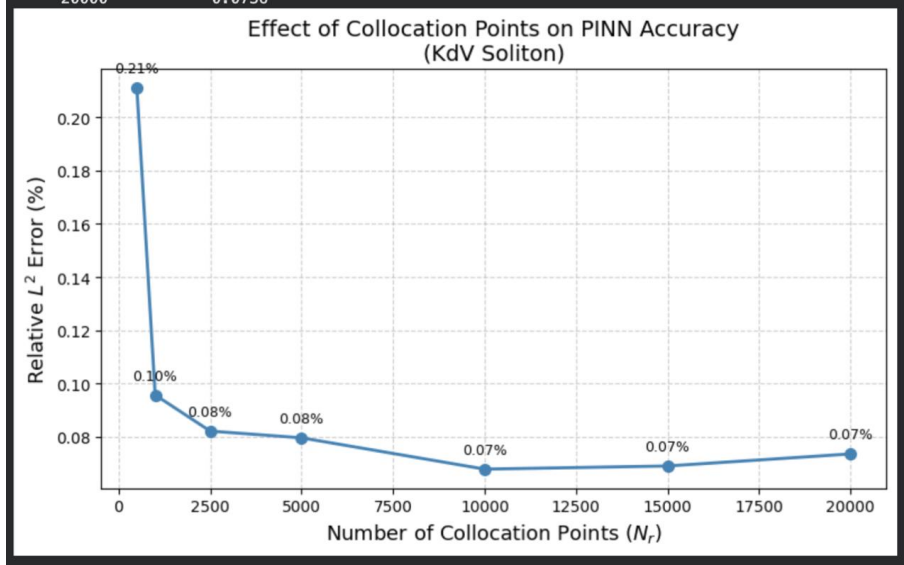


Figure 11: Saturation curve: error drops sharply up to $N_r \approx 5,000$ and then flattens. The slight uptick at 20,000 suggests over-dense sampling is mildly counterproductive.

What we learned:

There are diminishing returns beyond about 5,000 points. Even 500 collocation points give 0.21% error, which is already good. The slight increase at $N_r = 20,000$ is worth noting—it likely reflects the optimizer spending capacity on the PDE loss in flat, featureless regions at the expense of IC and BC accuracy. The message: **you do not need to flood the domain with collocation points**. A few thousand is enough for this class of problem, and more can actually hurt.

5.3 Experiment 3: Can We Drop the Initial Condition?

In real problems, you often do not know the initial state of the system. So we removed $\mathcal{L}_{IC}$ entirely and trained with PDE + BC + 200 clean data points only.

Table 5: Effect of removing the initial condition.

Configuration	After Adam	After L-BFGS
With IC (full PINN)	0.0717%	0.0717%
Without IC	0.2263%	0.0767%

What we learned:

The final error without IC (0.077%) is almost identical to the baseline with IC (0.072%). The 200 data points scattered across the domain implicitly contain information about the initial profile—some of them land near $t = 0$ and anchor the starting shape. The PDE and boundary conditions do the rest.

This is a practically important result. It says: **if you have even sparse sensor coverage across your domain, you may not need to know the starting state**. The PDE fills in what the sensors miss.

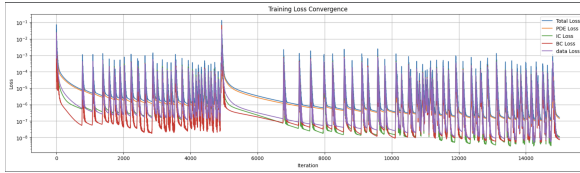
Notice also that L-BFGS helps here (unlike Experiment 1). The key difference is that the data is clean—there is no gradient conflict between the PDE and data losses. L-BFGS does what it is supposed to do: converge precisely to the right answer.

5.4 Experiment 4: Faster and Slower Waves

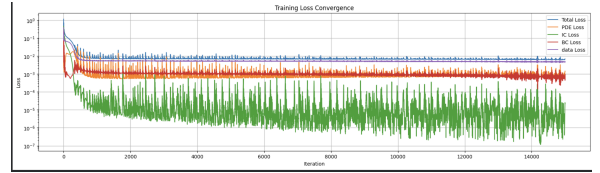
We trained the same architecture on $c = 1$ (slower, shallower soliton) and $c = 4$ (faster, steeper soliton), using the same domain $[-10, 20] \times [0, 5]$.

Table 6: Performance at different wave speeds. All use the same architecture and domain.

c	Peak depth	Soliton at $t = 5$	After Adam	After L-BFGS
1	-0.5	$x = 5$	0.073%	0.071%
2	-1.0	$x = 10$	0.072%	0.072%
4	-2.0	$x = 20$	29.5%	29.4%



(a) $c = 1$: loss reaches $\sim 10^{-6}$.



(b) $c = 4$: loss stalls at $\sim 10^{-2}$.

Figure 12: Loss convergence for $c = 1$ and $c = 4$. The same architecture that works beautifully at $c = 1$ completely fails at $c = 4$.

What we learned:

$c = 1$ and $c = 2$ work equally well. $c = 4$ fails badly, for two reasons:

1. The soliton at $c = 4$ has peak amplitude -2 and is proportionally narrower. The gradients are twice as steep, and the 6×50 network does not have enough capacity to resolve them.
2. At $c = 4$, the soliton reaches $x = 20$ exactly at $t = 5$, so it hits the boundary. The zero-boundary assumption (Eq. 4) is violated.

The fix is not complicated—extend the domain to $[-10, 30]$ or shorten the time to $[0, 3]$ —but the failure itself is instructive. **PINN accuracy is not architecture-universal.** The domain and network must be sized to the solution’s sharpness and speed.

6 Discussion: What This Tells Us About Using PINNs

Our four experiments, taken together, paint a consistent picture of what matters when deploying PINNs for wave equations.

The PDE does the heavy lifting. Experiment 1 shows it denoises. Experiment 2 shows even sparse enforcement ($N_r = 500$) captures the solution to 0.2%. Experiment 3 shows it can compensate for a missing initial condition. The PDE is not just a regulariser, it is the

backbone of the method. If you know the governing equation well, you need surprisingly little data.

The optimizer matters more than people think. The standard PINN training recipe is to run Adam for coarse convergence, then switch to L-BFGS for fine-tuning. For clean data, this works well—L-BFGS uses curvature information to take large, precise steps toward the minimum. But Experiment 1 revealed that when the 200 data points carry noise, L-BFGS actually *increases* the PINN’s error (e.g., from 5.5% to 6.8% at 5% noise). The reason is a gradient conflict: the PDE loss pushes the network toward smooth, physically consistent solutions, while the noisy data loss pushes it toward hitting corrupted targets. Adam, being a cautious first-order method, balances these competing signals with small steps. L-BFGS, being a second-order method, charges aggressively toward minimising both simultaneously—and ends up overfitting the noise. Curiously, the data-only model (which has no PDE loss) does *not* suffer from this: without a competing physics term, there is no gradient conflict for L-BFGS to mishandle. This asymmetry—L-BFGS hurting PINNs but not pure data-fitting models under noise—has not been widely reported, and it means the training strategy should be chosen based on the expected data quality, not applied as a fixed recipe.

Know your problem’s difficulty before choosing your architecture. Experiment 4 showed that our 6×50 network solved the $c = 1$ and $c = 2$ solitons to $< 0.08\%$ error, but failed at $c = 4$ with 29.5% error. The $c = 4$ soliton has twice the peak amplitude (-2 vs -1) and a proportionally narrower profile, meaning the spatial gradients are much steeper. Additionally, at $c = 4$ the wave travels fast enough to reach the right boundary ($x = 20$) by $t = 5$, violating the zero-boundary assumption that the network was trained on. The lesson is that PINNs do not automatically adapt to harder physics—a network that works for one wave speed may fail for another. The domain must be wide enough that the wave never interacts with the boundary, and the network must be deep or wide enough to resolve the sharpest features in the solution.

7 Conclusion

We solved the KdV soliton with a PINN to 0.07% accuracy, then ran four experiments that produced actionable findings for anyone using PINNs on wave-like PDEs:

1. The PDE residual cuts noise-induced error by $4.5\times$ at 10% noise. If you know the physics, use it.
2. PDE enforcement saturates around 5,000 collocation points. More is wasteful.
3. 200 scattered measurements can replace the initial condition entirely. You do not always need to know the starting state.
4. The network and domain must be scaled to the solution’s sharpness. $c = 4$ fails where $c = 2$ succeeds—same architecture, same code, different physics.

The unexpected finding—that L-BFGS hurts PINNs under noisy data—is perhaps the most practically relevant. It means the standard training recipe needs to be modified for real-world applications where measurement noise is unavoidable.

Future work. Natural next steps include two-soliton collisions, inverse problems (recovering c from sparse data), adaptive loss weighting for noisy settings, and architecture scaling for steeper solutions.

References

- [1] N. Schälch, *The Korteweg-de Vries Equation*, ETH Zürich Proseminar, March 2018.
- [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs,” *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.
- [3] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *ICLR*, 2015.
- [4] P. G. Drazin and R. S. Johnson, *Solitons: An Introduction*, Cambridge University Press, 1989.